ECE 315 Lab 2 Report

Miro Straszynski
Ebuka Odeluga

ECE 315 — Section H21
March 10, 2026

Abstract

This report presents the design and implementation of UART-based communication on the Zybo embedded development board. The project is divided into three main stages. First, a polling-based UART application is implemented that receives user input consisting of a username and password, processes the data, and generates a secure hash using the SHA-256 algorithm. The system demonstrates how UART peripherals can be configured, initialized, and interfaced with using a polling loop, while transmitting processed data through a queue-based mechanism.

In the second stage, the application developed in a previous laboratory exercise is extended by integrating UART communication into the system. This enables command input through UART to control various board peripherals, demonstrating how serial communication can be incorporated into an existing embedded system to improve interaction and control.

Finally, an interrupt-driven UART driver is developed to replace the polling-based approach. By using interrupts, the system can respond asynchronously to incoming data without requiring continuous CPU monitoring, improving efficiency and responsiveness. This progression highlights the advantages and trade-offs between polling and interrupt-based communication while reinforcing key concepts in embedded system design and UART peripheral management.

Design

Part 1

This code implements Part 1 of the lab, which features configuring, initializing, and interfacing with a UART peripheral on the Zybo Z7 board using a polling loop. The application uses a polling loop to receive input from the UART device and sends that data to a queue, where it is received and processed, and a unique hash string is generated from the input.

The system diagram for part 1 is shown in Figure 1.

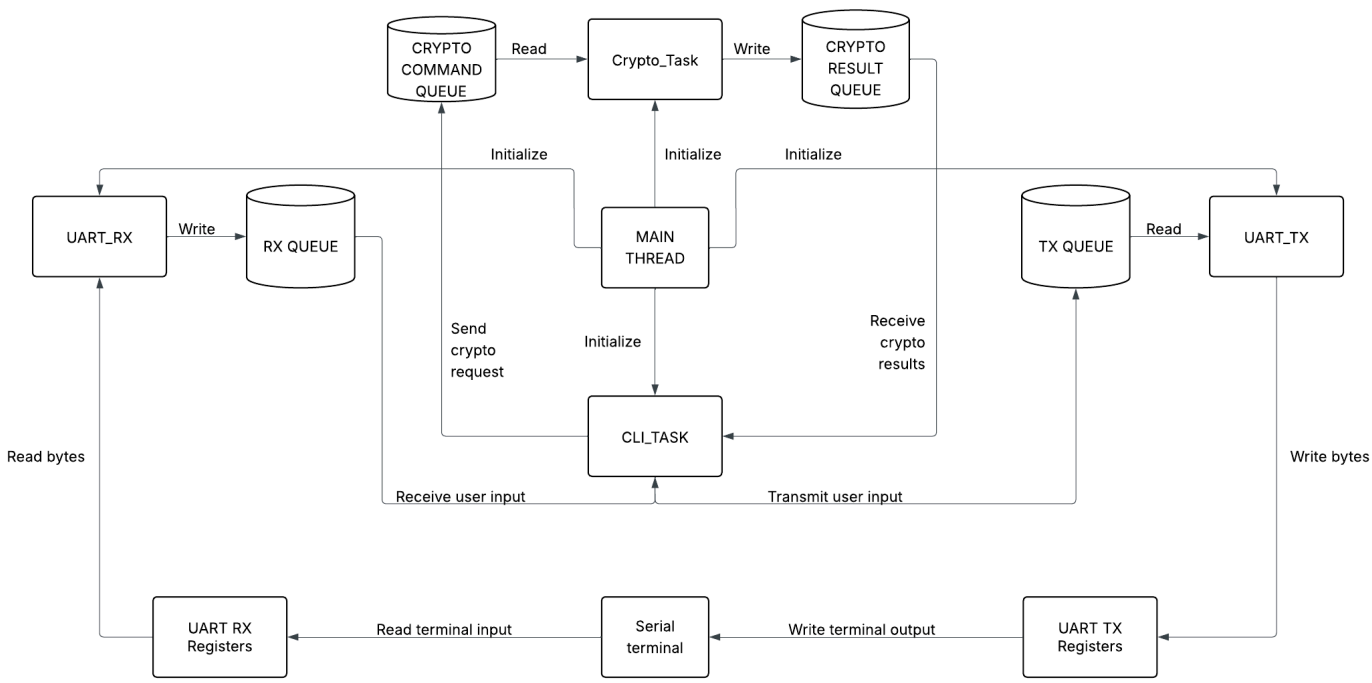

Figure 1: System Task Flow Diagram for Part 1.

When the application was launched before our implementation, an assertion error occurred. This was because the respective queues were not initialized, and the code contained assertions on the existence of these queues (e.g., configASSERT(q_rx_byte)). To fix this, we simply initialized the queues, and the assertion became true.

Busy waiting is when a processor continuously checks a condition in a loop until the condition becomes true. It was coined that term because the processor is being used (busy) while waiting for the condition to be true. This can be problematic as the processor is being used up when it could be performing other useful work. It is using up resources while waiting when it could be using it for other useful purposes.

Blocking operations are not appropriate in a polling-based design because polling requires the program to continuously check the status of devices and conditions in regular

intervals. Blocking operations cause the task or processor to stop execution and wait indefinitely for an event to occur, which defeats the entire purpose of polling, since the system can no longer regularly check the status of devices and conditions.

Questions

1. Why do we use non-blocking operations in polling-based systems?

Non-blocking operations are used in polling-based systems because they allow the program to check the status of a device or resource without stopping execution if the operation cannot be completed immediately. This ensures the system can continue polling other peripherals or tasks while waiting for an event, and prevents the CPU from becoming stuck waiting for a single operation to complete.

2. Why must a delay be included inside each polling loop?

A delay is included inside polling loops to prevent the CPU from continuously checking a condition at maximum speed, which would waste processing power. The delay reduces CPU usage and allows other tasks in the system—especially in an RTOS environment—to run.

3. How does the polling period affect system responsiveness and CPU usage?

The polling period directly affects the trade-off between responsiveness and CPU efficiency. A short polling period allows the system to detect events quickly, but increases CPU usage because the system checks more frequently. A long polling period reduces CPU usage but increases response time, since the system may take longer to detect when an event occurs.

4. What problems can arise if a polling loop is implemented incorrectly?

If a polling loop is implemented incorrectly, it can lead to excessive CPU usage, delayed task execution, missed events, or system unresponsiveness. For example, failing to include delays can cause busy-waiting that monopolizes CPU resources, while poorly designed polling logic may result in race conditions or lost data if events occur faster than the polling interval.

5. How can you determine an appropriate polling period for your system?

An appropriate polling period is determined by considering the timing requirements of the system, the speed at which events occur, and acceptable response latency. The polling interval should be short enough to detect events within the required response time but long enough to avoid unnecessary CPU usage. Testing, system profiling, and analyzing peripheral timing specifications are commonly used to find a suitable balance.

6. What happens if a high-priority task busy-waits in a FreeRTOS system?

If a high-priority task busy-waits in a FreeRTOS system, it continuously consumes CPU time without yielding control. Because FreeRTOS schedules tasks based on priority, this can starve lower-priority tasks, preventing them from running and potentially causing system instability or missed deadlines. Proper task design should use delays, blocking calls, or synchronization mechanisms to allow the scheduler to allocate CPU time fairly.

Part 2

In this part of the lab, we return to the code from Part 3 of Lab 1, this time implementing a UART command system, enabling the user to change the brightness and color of the RGB LED, and update the seven segment display through the serial console. To facilitate the sending and receiving of commands, we defined a custom command struct, which holds an enum relating the type of command, and a union of possible parameters, which is dependent on the command type. The union allows the command structure to be scalable, as the size of the struct will remain constant as long as the next member added to the union is no larger than the current largest member, which, as of the completion of part 2, is 2 bytes.

To parse a command, we wait for the 'n' or 'r' characters to be received, at which point the received command is sent to a helper function, parse_and_send. When parsing a command, we ignore whitespace and tab characters received, and expect the following characters to indicate the command being given: 'B' (brightness command), 'C' (color command), and 'S' (SSD command). For brightness, a value between 1-19 is accepted, and for the color command, we expect one of 8 characters corresponding to a possible color of the RGB LED. For the SSD command, we expect two space separated characters, which can be displayed by the SSD.

After parsing and building up the command, it is sent to a queue for either the RGB LED task, or the SSD task. As such, we had to create two new queues, one for RGB commands, and one for SSD commands. Each queue send has no wait time, and each of the queue receives in the receiving tasks have no delay, allowing us to avoid busy-waiting and let the tasks themselves determine their own polling delay.

Overall, the system works and appears responsive, with the original functionality still intact. The system diagram for part 2 is shown in Figure 2.

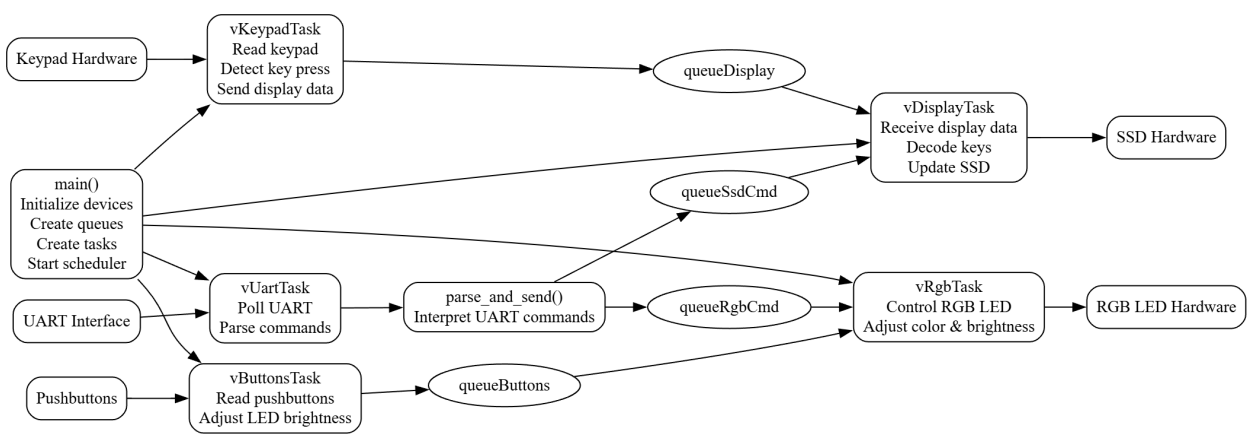

Figure 2: System Task Flow Diagram for Part 2.

Part 3

In this part of the lab, we implement an interrupt driven UART driver. A transmit and receive queue are used by the driver. When data arrives in the UART receive FIFO, a receive interrupt is triggered, and the interrupt service routine (ISR) places the read bytes into the receive queue. When bytes are transmitted, the data placed in the transmit queue is sent when the UART transmit FIFO is empty, at which point bytes are added until the FIFO is full or the queue becomes empty. A transmit-empty (TXEMPTY) interrupt is disabled/enabled based on the contents of the transmit queue. In this part of the lab, we implemented the functions mySendByte, mySendString, and myReceiveByte. In the sending functions, we found that it was necessary to write a byte to the UART FIFO before enabling the TXEMPTY interrupt, as this 'kickstarted' the queue emptying process.

The main system diagram for part 3 is shown in Figure 3. The task flow diagram for the UART driver is shown in Figure 4.

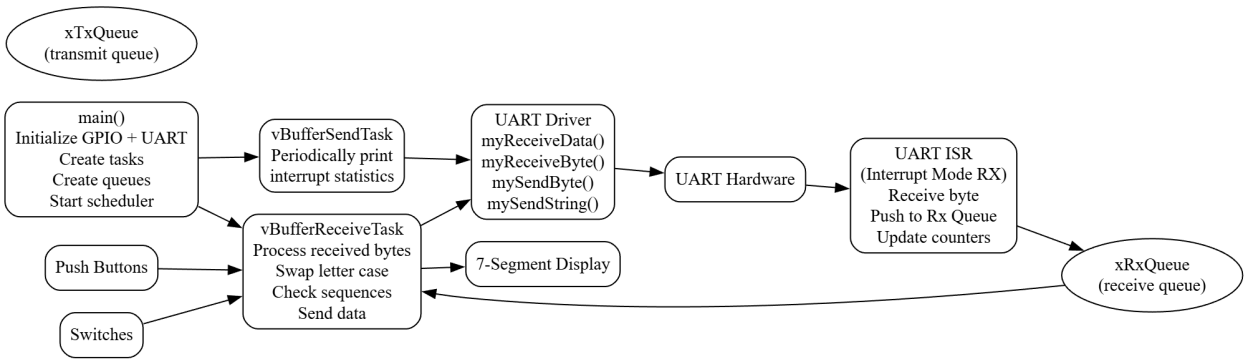

Figure 3: System Task Flow Diagram for Part 3.

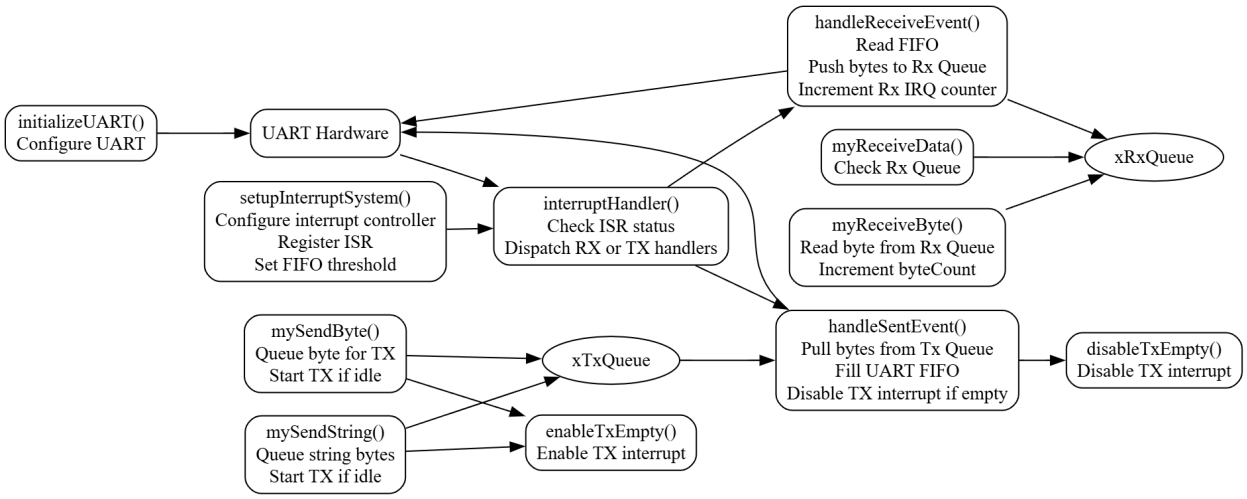

Figure 4: Task Flow Diagram for the UART Driver.

Questions

1. What are the implications for data integrity and system stability in the absence of protection for critical sections within the driver functions?

Without protection for critical sections, multiple tasks or interrupts may access shared data or hardware registers at the same time, leading to race conditions and data corruption. This can cause inconsistent or incorrect data to be transmitted or processed. Over time, such conflicts can also lead to unpredictable behavior and reduced system stability, including crashes or communication errors.

2. Is there a preferred order for processing interrupts? Briefly analyze how the sequence of handling transmit and receive interrupts in the interrupt service routine (ISR) impacts the system performance or reliability. Provide a rationale for why one sequence may be more advantageous over the other in the context of efficient system operation.

Yes, the order of handling interrupts can affect system performance and reliability. In many systems, receive (RX) interrupts are handled before transmit (TX) interrupts because incoming data must be read quickly to prevent buffer overflow and data loss. Prioritizing RX interrupts ensures data integrity, while TX operations can usually tolerate small delays without affecting system performance.

3. Explain the importance of disabling transmit interrupts when no data remains to be transmitted. What are the potential effects on the system if transmit interrupts were to remain enabled in such circumstances?

Disabling transmit interrupts when no data remains to be sent prevents the system from repeatedly triggering interrupts that have no useful work to perform. If transmit interrupts remain enabled, the processor may continuously enter the ISR even though there is no data to transmit, wasting CPU time and reducing system efficiency. This can also delay other tasks or interrupts, negatively impacting overall system performance and responsiveness.

4. Justify the number of interrupts that you found after sending three different-sized messages to the Zybo Z7 board and using both transmission methods (mySendByte and mySendString).

We sent 3 messages: 'a', 'asd', and 'hello'. Using mySendByte transmission method, 'a' had 2 interrupts, 'asd' had 4 interrupts, and 'hello' had 6 interrupts. This is because there was an interrupt for each character and then an interrupt for the newline or return character.

Using mySendString, 'a' still had 2 interrupts, 'asd' had 2 interrupts, and 'hello' also had 2 interrupts. This is because now there is an interrupt after the entire string, and an interrupt for the return character.

5. Discuss the importance of making queues and other implementation details private and inaccessible to the user in the context of the driver functions. Why is it important to hide implementation details from the users? How does encapsulating these details affect the overall system robustness and reliability?

Making queues and other implementation details private prevents users from directly modifying internal driver structures, which could lead to incorrect behavior or data corruption. By hiding these details, the driver ensures that all interactions occur through controlled interfaces. This encapsulation improves system robustness and reliability by reducing misuse and making the driver easier to maintain or modify.

Testing

Part 1

For part 1, testing involved using the given code and ensuring that the program worked as expected. We tested that the string hashing worked, and correctly processed the given string input. We also tested that incorrect hashes were rejected when the verify command was used. Test cases are shown in Table 1.

Test Case #	**Issued Commands**	**Expected Result**
1	‘1’ ; ‘hello’ ;	A hash is produced and is printed to the serial terminal.
2	‘2’ ; ‘hello’ ; [paste previously computed hash]	System prints that the calculated hash is the same as the expected hash.
3	‘2’ ; ‘hello’ ; ‘asdf’ (random string)	System prints that the calculated hash is not the same as the expected hash.

Table 1: Test cases for Part 1.

Part 2

In part 2, we followed a similar procedure as above for finding the maximum possible delay period in which flickering was no longer visible in the LED. As test cases follow the same format as above, it will be omitted. Here, we found the optimal delay to be 24 ticks.

We also tested the operation of the buttons to verify that they correctly changed the brightness of the LED, and verified that edge cases were covered. Example test cases are shown in Table 2.

Test Case #	**Issued Command**	**Expected Result**
1	B 19	LED changes to maximum brightness.
2	B 1	LED changes to minimum brightness.
3	B 0	LED remains at minimum brightness.
4	B 30	LED switches to maximum brightness (19).
5	C R	LED color changes to red.
6	C B	LED color changes to blue.
7	C A	LED color does not change from previous color.
8	S 1 2	Seven-segment display shows '12'.
9	S A B	Seven-segment display shows 'Ab'.
10	S X B	Seven-segment display shows ' b' (invalid character is ignored and side of SSD remains blank).

Table 2: Test cases for Part 2.

Additionally, all tests completed in part 3 of Lab 1 were repeated. These tests are omitted here for the sake of brevity. We ensured that the SSD still operated as expected, and that the RGB brightness continued to respond to pressing either of the buttons as expected.

Part 3

In part 3 of the lab, we primarily tested the UART driver by ensuring that the provided test/main program operated as expected. We tested by sending a simple string of 'hello' to the main program, and checking that the capitalized version was received correctly, and that the number of bytes transmitted and Rx interrupts were 8 total, and the number of Tx interrupts was 3, as expected. (7 bytes for hello + extra kickstarting byte). We continued to send other strings over UART to the case-switching program and each time checked that the number of bytes received and transmitted matched expectations. Test cases are shown in Table 3.

Test Case #	**Issued Commands**	**Expected Result**
1	'\r#\r'	0 received bytes, 0 transmitted bytes, and 0 interrupts.
2	'hello'	HELLO is repeated immediately following the input.
3	'hello' ; '\r#\r'	8 bytes received, 3 Tx interrupts, and 3 Rx interrupts.

Table 3: Test cases for Part 3.

Conclusion

In this lab, we successfully met the objectives in each of the parts. In part 1, we successfully configured and initialized the UART peripheral and validated its use in the provided hash generating program. In part 2 of the lab, the system created in lab 1 was successfully enhanced, adding a UART command interface. We found that this command interface successfully allowed sending commands to cause immediate effects in the seven-segment display. In part 3 of the lab, we were able to finish the implementation of interrupt-driven UART communication, and verified that the number of transmit and receive interrupts, as well as total number of bytes, was correct.

Throughout the lab, we had several hiccups come up. In part 1, we initially struggled to get output from the UART driver, having the first xil_printf replaced by print_string result in an infinite loop of outputting a part of the given string. It was discovered that this was because the size of the string index variable was only 2 bytes, and changing this to 4 bytes resolved the problem. In part 2, we mostly did not face any issues, but noticed that the seven segment display did not display digits in the same order as was sent through UART. This was easily resolved by changing the section of the SSD the left and right bytes were set to. In part 3 of the lab, we faced issues handling the Tx FIFO. This issue ended up being related to the fact that in our send byte/string functions, we hadn't been writing a byte to the FIFO to kickstart the ISR. Overall, these bumps were inconsequential and we were able to get each part working as expected.